Lenguaje Python

Cursada 2026

Clase 3: expresiones lambda - manejo de archivos

Pensemos en las siguientes situaciones

¿Qué estructura usamos si queremos:

- guardar los puntajes cada vez que jugamos a un juego determinado?,
- tener un banco de preguntas para que cada vez que realizamos el repaso de clase pueda acotar por temas?,
- manipular los Python plus de los estudiantes por turnos?

¿Qué tienen todas estas situaciones en común?

Necesitamos una estructura que permita que los datos puedan persistir cuando la ejecución del programa finalice.

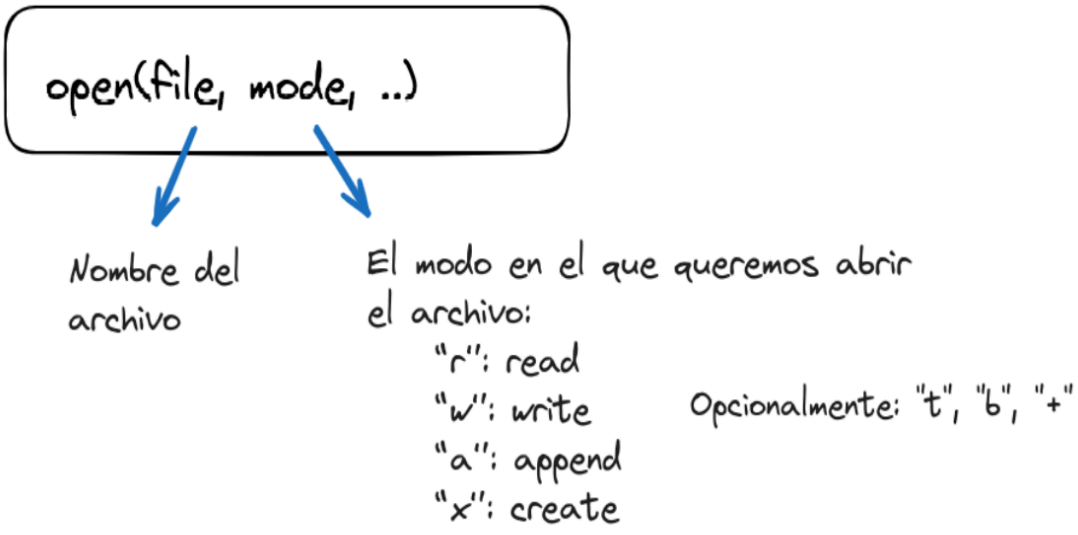
Algunas consideraciones antes de empezar

- Lo básico: ¿qué es un **archivo**?
- ¿Cómo podemos manipular los archivos desde un programa Python?

Manejo de archivos

- Existen funciones predefinidas.
- Si las operaciones fallan, se levanta una **excepción**.
- Los archivos se manejan como objetos que se crean usando la función open.

La función open



Veamos algunos ejemplos

Caso 1: usamos modo "w"

- ¿De **qué modo** se abre este archivo? ¿Qué significa?
- Luego de la instrucción, ¿**dónde se encuentra** archivo.txt?
- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
file1 = open('archivo.txt', 'w')
```

Caso 2: usamos modo "x"

- ¿De **qué modo** se abre este archivo? ¿Qué significa?
- Luego de la instrucción, ¿**dónde se encuentra** archivo.txt?
- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
file2 = open('archivo.txt', 'x')
```

Caso 3: usamos modo "a"

Veremos este caso más adelante...

Caso 4: ¿qué modo estamos usando?

- ¿De **qué modo** se abre este archivo? ¿Qué significa?

- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
file3 = open('archivo.txt')
```

En realidad la función open tiene más argumentos:

```
open(file, mode='r', buffering=-1, encoding=None,
     errors=None, newline=None, closefd=True, opener=None)
```

- **encoding**: sólo para modo texto. Por defecto, la codificación establecida en las configuraciones del sistema

```
notes = open("pp.xxx", "r+", encoding="UTF-8")
```

- En este caso, ¿debería llamarse pp.txt?
- ¿Cómo sé cuál es la configuración por defecto?

In [ ]:
```
import locale
locale.getlocale()
```

¿Qué pasa cuando el archivo está en otro directorio?

Usamos pathlib descripta en la PEP 428: una interfaz orientada a objetos que permite el manejo de rutas.

In [ ]:
```
from pathlib import Path
file_muchachos = Path('ejemplos') / "clase04" / "muchachos.txt"
file_muchachos
```

Archivos de texto vs. binarios

In [ ]:
```
the_best_song_file = open(file_muchachos,"rb")
lyrics = the_best_song_file.read()
the_best_song_file.close()
```

In [ ]:
```
print(lyrics)
```

In [ ]:
```
type(lyrics)
```

- El tipo **bytes** es una secuencia inmutable de bytes.
- Solo admiten caracteres ASCII.

In [ ]:
```
print(lyrics.decode('UTF-8')[:28])
```

- El tipo **str** de Python utiliza el estándar Unicode para representar caracteres.
- Generalmente la codificación predeterminada es **UTF-8**.
  - UTF significa **Unicode Transformation Format**, y el **8** significa que se utilizan valores de 8 bits en la codificación.

Una rosa azul ..

In [ ]:
```
file_rose = Path('ejemplos') / "clase04" / "rosa_azul.jpg"
rose = open(file_rose,"rb")
rose = rose.read()
```

In [ ]:
```
rose
```

¿Cómo almacenamos datos en un archivo?

El caso más sencillo: vamos a guardar solo texto plano.

In [ ]:
```
file_muchachos = Path('ejemplos') / "clase04" / "muchachos1.txt"
```

In [ ]:
```
f = open(file_muchachos, 'w')

cad1 = "De los pibes de Malvinas"
cad2 = "Que jamás olvidaré."

f.write("En Argentina nací")
f.write("Tierra del Diego y Lionel")
f.write(cad1)
print(f.write(cad2))

f.close()
```

- **write(cadena):** escribe *cadena* en el archivo y retorna cantidad de caracteres escritos.
- **close():** cierra el archivo.

Ahora si: abrimos con modo "a"

- ¿De **qué modo** se abre este archivo? ¿Qué significa?
- Luego de la instrucción, ¿**dónde se encuentra** archivo.txt?
- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
f = open(file_muchachos, "a")
f.write("******************************")
f.write("HOLA QUE TAL")
f.close()
```

¿Qué pasa si no existe?

In [ ]:
```
file_nuevo_muchachos = Path('ejemplos') / "clase04" / "muchachosNUEVO.txt
```

In [ ]:
```
f = open(file_nuevo_muchachos, "a")
```

¿Cómo accedemos a los datos guardados?

In [ ]:
```
f = open(file_muchachos, 'r')
x = f.read(4)
print(f.read())
x
```

- **read(cantidad_bytes):** lee *cantidad_bytes* del archivo.
- Si cantidad_bytes es <0 o no está, lee hasta fin de archivo.
- Retorna "" si EOF.

CONSIGNA: probar en casa

Analizar el siguiente ejemplo que muestra otras formas de leer caracteres desde un archivo de texto.

In [ ]:
```
file_muchachos = Path('ejemplos') / "clase04" / "muchachos.txt"
def by_characters():
    f = open(file_muchachos,"r")
    for x in f.read():
        print(x)
    f.close()

def by_lines():
    f = open(file_muchachos,"r")
    print(f.readlines())
    f.close()

def other_by_lines():
    f = open(file_muchachos,"r")
    for line in f:
        print(line)
    f.close()
```

In [ ]:
```python
def main():
    print('Leo caracteres')
    by_characters()
    print('-' * 20)
    print('Leo lineas')
    by_lines()
    print('-' * 20)
    print('Otra forma')
    other_by_lines()


if __name__ == "__main__":
    other_by_lines()
```

¿Se acuerdan de esto?

```python
if __name__ == "__main__":
    other_by_lines()
```

En resumen: - modos de apertura (1)

- **'r'**: modo de lectura (**read**). Abre un archivo para lectura. **El modo por defecto, por lo que puede omitirse.**
  - Si el archivo **no existe**, se genera un error **FileNotFoundError**.
  - **Error**: intentar abrir un archivo que no existe o sin permisos.
- **'w'**: modo de escritura (**write)**. Abre un archivo para escritura.
  - Si el archivo **ya existe, se trunca** (se borra su contenido).
  - **Si no existe, se crea uno nuevo**.
  - **Error**: si no tengo permisos de escritura. Si no es el caso, no hay un error específico al abrir, pero se perderá el contenido del archivo si ya existía.

En resumen: - modos de apertura (2)

- **'a'**: modo para agregar (**append**). Abre un archivo para **agregar contenido al final**.
  - Si el archivo **no existe, se crea uno nuevo**.
  - **Error**: si el archivo **no tiene permisos de escritura**, se generará un error **PermissionError**.
- **'x'**: modo de **creación**. Abre un archivo para escritura, pero **solo si no existe**.
  - Si el archivo **ya existe**, se genera un error **FileExistsError**.
  - **Error**: intentar abrir un archivo que ya existe o si no tenemos permisos de escritura.

En resumen: - modos de apertura (3)

- **'r+'**: modo de lectura y escritura (read and write). Abre un archivo para lectura y escritura. **El archivo debe existir**; de lo contrario, se genera un error **FileNotFoundError**.
- **'w+'**: modo de lectura y escritura (write and read). Abre un archivo para lectura y escritura.
  - **Si el archivo ya existe**, se trunca.
  - **Si no existe**, se crea uno nuevo.
- **'a+'**: modo para agregar y lectura (append and read). Abre un archivo para agregar **contenido al final** y también permite la lectura.
  - Si el archivo **no existe**, se crea uno nuevo.
  - Si el archivo **no tiene permisos de escritura**, se generará un error **PermissionError**
- **'x+**: modo de creación. Crea un archivo para lectura y escritura. El archivo no debe existir.
  - Si el archivo **ya existe**, se genera un error **FileExistsError**.

En resumen: tipos de archivos

- **'b'**: **binario** (binary). Se utiliza al especificar el modo de apertura (por ejemplo, 'rb', 'wb', 'ab') para abrir archivos en formato binario.
- **'t'**: **texto** (text). Se utiliza al especificar el modo de apertura (por ejemplo, 'rt', 'wt', 'at') para abrir archivos en formato texto. **Es el formato por defecto, por lo que puede omitirse.**

DESAFIO: retomamos el primer ejemplo del repaso 1

Problema:

> Tenemos que procesar los datos de los estudiantes de las tres comisiones de Python. Por cada estudiante contamos con su DNI, nombre, comisión, puntos de teoría, puntos de práctica y ciudad de procedencia.

¿Qué opciones tenemos para almacenar estos datos en archivos de texto?

Opción 1

```
'DNI': 12345, 'name': 'Guido Van Rossum', 'pts_teoria':
100, 'pts_practica': 50, 'city': 'Haarlem', 'group':
'info:TM'
'DNI': 11111, 'name': 'Ada Lovelace', 'pts_teoria': 100,
'pts_practica': 50, 'city': 'Londres', 'group': 'info:TM'
'DNI': 12345, 'name': 'Alan Turing', 'pts_teoria': 100,
'pts_practica': 50, 'city': 'Londres', 'group': 'info_TT'
```

```
'DNI': 11111, 'name': 'Joan Clarke', 'pts_teoria': 100,
'pts_practica': 50, 'city': 'Londres', 'group': 'info_TT'
```

Opción 2

```
'DNI': 12345
'name': 'Guido Van Rossum'
'pts_teoria': 100
'pts_practica': 50
'city': 'Haarlem'
'group': 'info:TM'
****
'DNI': 11111
```

CONSIGNA: realizar en casa

> Escribir un programa que almacene los datos del ejemplo en un archivo de texto.

NOTA: no usar ninguna librería adicional.

In [ ]:
```
#Posible solución
```

Seguimos la semana próxima ...